Speech Flaw Analyzer: Technical Documentation

Team: Caps Lock Crew (4 members)

Member	Role
Ayush Dattaram Bangal	Team Leader
Aditya Raj Singh	Team member
Khushi	Team member
Shubham Kanyal	Team member

1. Overview

Speech Flaw Analyzer compares a speech reading with a reference reading of the same text and finds the time regions where delivery deviates: rushed pace, long pause and volume drop. Each flagged region comes with a measured reason, for example "word duration 0.67x of this speaker's usual pace".

The system is a rule-based pipeline with no trained classifier, so every flag traces back to a feature difference. The only learned component is the speech recognizer, which supplies word timestamps. A Streamlit dashboard lets a user upload or record audio and see the flagged regions as red bands with timestamps, reasons and playback.

Track: C, Contrastive Speech Analytics. **Code, dataset and demo video:** see the project README on GitHub.

2. Dataset construction

The dataset pairs clean readings with programmatically flawed audio, plus transcripts and word-level alignment labels. All of it is in the GitHub repository.

Text. The baseline text is the first two paragraphs of the Gettysburg Address (1863, public domain), stored in `data/baseline/speech1.txt`.

Recording. Two speakers, A and B, each read the text on a phone in a quiet room. The raw files (`A_raw.ogg`, `B_raw.mp4`) were converted to WAV with ffmpeg.

Alignment. Word timestamps come from faster-whisper (`small` model, int8, CPU), with audio loaded through librosa at 16 kHz mono. Each word is stored as `{word, start, end}` in `labels/*_words.json`. The raw recognizer output is kept in `*_words_asr.json`.

Features. For every word, `features.py` stores duration, the silence before the word, loudness in dB, and pitch (median and range in semitones relative to the speaker's own median, from Praat via parselmouth). Using each speaker's own median makes different voices comparable.

Flaw injection. `flaw_injector.py` creates controlled flaws from a recording and writes a label file for each one:

Flaw	How it is made	Injected region (test clip)
Rushed pace	A section is time-stretched to 1.5x speed	3.0-6.333 s
Long pause	2.5 s of silence is inserted	5.54-8.04 s
Volume drop	Gain is reduced over a section	3.0-8.0 s

Each label stores the flaw type and its start and end time on both the original and the modified timeline. Both are needed because stretching or inserting audio shifts everything after it. Labels are written by the script, so they contain no typing errors.

Scope. The injected-flaw files come from a 15 s test clip, not from the Gettysburg readings. There is one severity level per flaw type.

3. Model architecture

The system is a four-stage, rule-based pipeline with no trained classifier.

1. **Alignment** (`align.py`): faster-whisper produces word start and end times.
2. **Feature extraction** (`features.py`): per-word duration, gap before the word, loudness and pitch.
3. **Comparison** (`detect.py`): the recording is matched to the baseline word by word, and deviating words are grouped into regions.
4. **Dashboard** (`app.py` , Streamlit): the user uploads or records audio, and the app shows red bands on the timeline, timestamps, the reason text and playback of each flagged region.

Why rule-based. Every flag can be explained with the numbers behind it, so a user can see why a region was flagged and check it by ear. A learned scorer would need labelled real-world flaws, which this project does not have, and would be harder to explain. The speech recognizer is the only learned part, and it is used only for timestamps.

Reference and free-speech modes. The main mode compares a reading with a reference reading of the same text. A second mode, free speech (`detect_solo.py`), compares a speaker with their own average. It is experimental and not validated.

4. Scoring methodology

The detector (`detect.py`) applies fixed rules to word-level feature differences. It produces flagged regions, not a single numeric score.

Step 1: word matching. Words are lowercased and reduced to letters and digits. The two word sequences are matched with Python's `difflib.SequenceMatcher`, and only words found in both are compared. This tolerates recognition errors and skipped words.

Step 2: speaker normalisation. Two corrections are computed per recording, so a different voice, or a generally slower or quieter reading, is not flagged as a whole.

- *Tempo*: the median ratio of word duration (recording / baseline) over matched words of at least 0.05 s.
- *Level*: the median loudness difference (baseline minus recording, in dB) over matched words.

Step 3: flag rules.

Flaw	Rule	Threshold
Rushed pace	Tempo-corrected duration ratio is below 1 minus 0.25	ratio < 0.75
Long pause	Silence before a word exceeds the baseline's silence before the same word	more than 0.8 s longer
Volume drop	Per-word drop (baseline minus recording, minus the level correction) averaged over a sliding window of 6 words	average above 9 dB; averages of 40 dB or more are discarded as artifacts

Volume is judged over a 6-word window because single-word loudness is noisy.

Step 4: region building. Flags of the same type that are less than 1.5 s apart are merged into one region, and the reason of each flag is kept. A rushed-pace region needs at least 3 flagged words, because one fast word is usually a timestamp error.

Output. Each region has a type, a start, an end and a list of measured reasons. The rules are deterministic, so the same inputs always give the same output, which was verified by running the evaluation twice. The thresholds were set by hand on the 15 s test clip and were not tuned on a held-out set.

5. Evaluation rubric and results

The detector finds all three injected flaws with IoU between 0.88 and 0.94, but it also flags regions on clean speech, so flags on real recordings should be read with caution.

Rubric (`evaluate.py`)

- **IoU** (intersection over union) is the overlap between a detected region and the injected region, divided by their combined span. 1.0 is a perfect match and 0 is no overlap.
- A flaw counts as **detected** if a region of the same type has IoU above 0.3.
- A **false alarm** is a detected region that overlaps no injected flaw of the same type.
- **Reproducibility** is checked by running the evaluation twice and comparing the outputs.

Test 1: injected flaws (`labels/eval_results.json`)

Flaw	Injected region (s)	IoU	Detected
Rushed pace	3.0-6.333	0.924	Yes
Long pause	5.54-8.04	0.88	Yes
Volume drop	3.0-8.0	0.943	Yes

False alarms: 1. In the pause file the detector also flagged a volume drop from 3.6 to 10.46 s that was not injected. A likely cause, not yet verified, is that the inserted silence shifts the file's loudness statistics. Two evaluation runs gave identical output.

Test 2: clean speech false-alarm test (`labels/ab_results.txt`)

Speaker B's clean reading was compared against speaker A's clean reading. A perfect detector would flag nothing. The result file lists 9 flagged regions: 8 volume drops and 1 rushed-pace region (32.48-36.38 s). Several of the volume reasons report 20 to 25 dB drops, which looks like word timestamps covering silence or breaths rather than real quiet speech.

Reading the results. Test 1 is the easiest case, because the baseline and the flawed file come from the same recording. Test 2 is the harder and more realistic case, and it shows the detector over-flags volume on real speech.

6. Limitations

- **Easy injected test.** The injected-flaw files come from the same 15 s recording as their baseline, which is the easiest case. They were not made from the Gettysburg readings.
- **False alarms on clean speech.** Comparing two clean readings flagged 9 regions, mostly volume drops. Detection of injected flaws is reliable, but real-speech flags need human review.
- **Silence side effect.** Inserting a long silence can trigger a false volume-drop flag on the words after it.
- **Approximate timing.** Word times come from speech recognition and were within about 0.2 s in a manual spot-check against the waveform.
- **Hand-set thresholds.** Thresholds were chosen by hand on one clip and not tuned or validated on a held-out set.
- **Narrow coverage.** Only three flaw types and one severity level are covered, and there is no mild level.
- **Free speech mode.** Comparing a speaker with their own average is experimental and not validated. The reference-comparison mode is the main result.